

Fall 2026 Exploratory Data Analysis with R

Lecture 4: Code Hygiene and R Markdown

Brian Robertson
Federal Reserve Bank of Atlanta
2026-09-18

01

Recap & Agenda

01 Recap & Agenda

02 Code Hygiene

03 Using the Right Function for the Job

04 Debugging

05 Introduction to R Markdown

06 Recap

Quick Recap: Last Week

- Matched the **summary statistic** to the **variable type** — continuous/discrete vs. categorical
- Measured **location** (mean, median, mode) and **spread** (sd, variance, range, IQR)
- Read many statistics at once with `summary()` and `sumtable()`
- Computed **conditional** summary statistics with `group_by()` + `summarize()`
- Learned to **bin a continuous variable** so we can group by it

Today: less about *new* statistics, more about writing code that you — and everyone reading it — can actually follow.

Goals for Today

- Establish **code hygiene**: organizing, commenting, and naming so your scripts stay readable
- Learn to reach for the **right function for the job** — and meet `case_when()` for multi-way conditions
- Pick up basic **debugging**: reading error messages and isolating a broken pipe
- Introduce **R Markdown** for reproducible write-ups that weave together text, code, and results
- Build tables for a report with `kable()`

Getting Started

1. Fire up R in your Posit accounts (or your IDE of choice) and start a new project
2. Prepare your environment:

```
install.packages('tidyverse')
install.packages('AER')
install.packages('janitor')
install.packages('knitr')
install.packages('kableExtra')

library(tidyverse)
library(AER)
library(janitor)
library(knitr)
library(kableExtra)

# BankWages ships with the AER package, so we load it with data()
data("BankWages")
```

02

Code Hygiene

01 Recap & Agenda

02 Code Hygiene

03 Using the Right Function for the Job

04 Debugging

05 Introduction to R Markdown

06 Recap

Why Code Hygiene?

Your R script always runs **top to bottom**. Organized code is easier to read, easier to hand off, and far less likely to break.

Think of a script the way you'd think of writing:

- An R **script** is your scrap paper — where you draft, test, and make mistakes.
- An R Markdown document (later today) is your **final draft** — the clean version a reader sees.

Code hygiene is just the habits that keep the scrap paper legible enough to become a final draft.

An “Order of Operations” for a Script

A good script reads in a predictable order, top to bottom:

1. A brief comment saying **what the script does** — its inputs (data) and outputs (tables, plots)
2. Load packages with `library()`
3. If useful, store repeated **file paths** in variables (e.g. a data folder, a plot folder)
4. **Read in** your data
5. **Section** the rest of the code with comments

Same idea as last week’s tidy workflow: a reader should be able to follow your script without you standing over their shoulder.

Commenting

A **comment** is text in a code file that R does not run. In R, anything after a `#` on that line is a comment:

```
# Load and attach packages  
library(tidyverse)
```

- There is no multi-line comment in R, but in RStudio you can comment a block with **Ctrl+Shift+C** (Cmd+Shift+C on a Mac).
- A comment line ending in four or more `-`, `=`, or `#` becomes a foldable **code section** — a quick way to organize a long script.

Comment the *Why*, Not the Obvious

Good comments explain **intent** — why the code exists, or how a tricky step works.

- Leave a comment at the **top** of every script: what it does, its inputs and outputs, and anything subtle.
- Above a function you write, describe its inputs (parameters) and what it returns.
- A comment can be a one-liner — but a *descriptive* one-liner.

You can over-comment. Don't narrate self-explanatory lines:

```
# Increment counter by 1  
counter <- counter + 1
```

Commenting Out Code

It's called **Exploratory** Data Analysis for a reason — not every line survives. Commenting code out (rather than deleting it) lets you:

- keep a dead end *just in case* you want it back, and
- isolate a bug by switching lines off one at a time.

```
# Parked for later: sales vs. crash rating
# data <- mtcars %>%
#   left_join(country_of_origin) %>%
#   group_by(crash_rating) %>%
#   summarize(avg_price = mean(sales_price))

# Add a kilometers-per-gallon variable
data <- mtcars %>%
  # mutate(kpg = mpg * 1.60934) # old name, kept for reference
  mutate(kpg = as.numeric(mpg) * 1.60934)
```

Variable Names

Names are documentation. A good name tells a reader what a variable *is* without a comment.

- Keep them **short but descriptive**
- Make them **searchable** — you'll want to find every use later
- Separate words with `snake_case` or `camelCase`

```
# Hard to search, tells you nothing
var1 <- starwars$name
var2 <- starwars$films

# Names that explain themselves
sw_characters <- starwars$name
sw_films      <- starwars$films
```

Test Before You Overwrite

Before you make a **permanent** change to a data frame, try it on a temporary copy first. If it breaks, you debug the copy — not the original.

```
data <- mtcars

# Test on a copy — quick, one-off checks can also live in the console
data_temp <- data
data_temp <- data_temp %>%
  mutate(kpg = mpg * 1.60934) # kpg = kilometers per gallon

# Once it works as intended, write it back
data <- data_temp
```

This is the *scrap paper* idea again: prove the code works before you commit to it.

03

Using the Right Function for the Job

- 01 Recap & Agenda
- 02 Code Hygiene
- 03 Using the Right Function for the Job**
- 04 Debugging
- 05 Introduction to R Markdown
- 06 Recap

The Right Function for the Job

Readable code isn't only good names and comments — it's picking the **function built for the task**. The wrong tool works but reads badly; the right one often disappears into a single, obvious line.

Last week we flagged a **binary** condition with `if_else()`:

```
mutate(age_group = if_else(age >= mean(age), "older", "younger"))
```

One test, two outcomes. But what happens when there are **more than two** categories?

Meet BankWages

`BankWages` (from **AER**) records the `job`, years of `education`, `gender`, and `minority` status for 474 bank employees.

```
glimpse(BankWages)
```

```
Rows: 474
Columns: 4
$ job      <ord> manage, admin, admin, admin, admin, admin, admin, admin, adm...
$ education <int> 15, 16, 12, 8, 15, 15, 15, 12, 15, 12, 16, 8, 15, 15, 12, 12...
$ gender    <fct> male, male, female, female, male, male, male, female, female...
$ minority  <fct> no, no, no, no, no, no, no, no, no, no, no, yes, yes, yes, n...
```

`education` is **discrete** (years of schooling). Suppose we want to group employees by **highest degree attained** — several buckets, not two.

Our Categories

Using `education` (in years), we'll define **degree attainment**:

- `education < 12` → **Less than HS**
- `education == 12` → **High school**
- `12 < education < 16` → **Some college**
- `education == 16` → **Bachelor's**
- `education >= 17` → **Graduate**

Five outcomes. `if_else()` only gives us two — so how do we express this?

The Nested `if_else` Trap

You can stack `if_else()` inside `if_else()`, one per split:

```
BankWages %>%  
  mutate(degree = if_else(education < 12, "Less than HS",  
    if_else(education == 12, "High school",  
      if_else(education < 16, "Some college",  
        if_else(education == 16, "Bachelor's",  
          "Graduate")))))
```

It runs — but count the parentheses. Every new category nests one level deeper, the logic reads inside-out, and a single missing `)` breaks it all. This is the “endless sea of nested `if_else`.”

case_when(): One Test Per Line

`case_when()` is the right tool for **many mutually exclusive conditions**. Each line is `condition ~ value`, read top to bottom:

```
BankWages <- BankWages %>%  
  mutate(degree = case_when(  
    education < 12 ~ "Less than HS",  
    education == 12 ~ "High school",  
    education > 12 & education < 16 ~ "Some college",  
    education == 16 ~ "Bachelor's",  
    education >= 17 ~ "Graduate"))
```

Flat, aligned, and it reads like the table on the earlier slide. Adding a sixth category is one more line — no extra nesting.

Order Matters – and Add Structure

`case_when()` checks conditions **top to bottom** and stops at the first match, so order your rules from most to least specific.

To make the categories a *true* ordered category, wrap the result in `factor()`:

```
BankWages <- BankWages %>%  
  mutate(degree = factor(degree,  
    levels = c("Less than HS", "High school",  
               "Some college", "Bachelor's", "Graduate"),  
    ordered = TRUE))
```

Now `degree` sorts in a sensible order in every table and plot — just like the factors from Lecture 3.

Putting It to Work

A cross-tab of `job` against our new `degree` — the payoff of getting the categories right:

```
BankWages %>%
  tabyl(job, degree) %>%
  adorn_title(placement = "combined")
```

job/degree	Less than HS	High school	Some college	Bachelor's	Graduate
custodial	13	13	1	0	0
admin	40	176	117	24	6
manage	0	1	4	35	44

What is the most common degree attainment among management jobs?

Breakout Session 1

Task: Bank employees fall into three `job` families. Build a categorical variable that groups **years of education** into your own buckets, then summarize.

1. Use `mutate()` with `case_when()` to create a variable `ed_level` with **three** categories of your choosing (e.g. "HS or less", "Some college", "College+").
2. `group_by(job)` and `summarize()` the **mean** education and the **count** (`n()`) of employees.
3. Which `job` family has the highest average education? Which is the largest share of the sample?

Hint: conditions are checked top to bottom — write them so the categories don't overlap.

Reaching for the Right Tool

`case_when()` is one example of a general habit: **match the function to the task**. The readable choice is usually the one written for exactly what you're doing.

The task	Reach for
One condition, two outcomes	<code>if_else()</code>
Many mutually exclusive conditions	<code>case_when()</code>
Count categories / cross-tabulate	<code>tabyl()</code> , <code>count()</code>
Summaries by group	<code>group_by()</code> + <code>summarize()</code>

The nested-`if_else` version *worked* — but the right function made the code shorter, flatter, and obvious to a reader. That is code hygiene.

04

Debugging

- 01 Recap & Agenda
- 02 Code Hygiene
- 03 Using the Right Function for the Job
- 04 Debugging**
- 05 Introduction to R Markdown
- 06 Recap

Debugging a Pipe

A long `%>%` chain that throws an error doesn't tell you *which* step broke.

- **Reverse-engineer it:** run the chain one line at a time, adding a step until it breaks. The line you just added is the culprit.
- Remember what the pipe does: it feeds the output of one function in as the **first argument** of the next. If a step gets the wrong input, look one line up.

Reading Error Messages

Always read the message first — on a good day it names the problem and points at the fix. Common causes:

- **Syntax** — a missing comma, an extra bracket, a typo, wrong capitalization
- A function from a package you **haven't loaded** with `library()`
- Data in the **wrong format** for the function

Still stuck on the message?

- Type `?function_name` in the console for its help page
- Search the exact error text online — someone has hit it before
- See the **How to Ask a Question** guide on Canvas

Warnings Are Not Errors

- A **warning** means R kept going, but *something may be off* — the result can still be wrong. Read it; don't reflexively ignore it.
- An **error** means R could not continue and execution stopped.

A script that runs without errors is not the same as a script that runs **correctly** — warnings and sensible-looking-but-wrong output are exactly why we test on copies and check our summary statistics.

Breakout Session 2

Task: the chain below is meant to report mean `education` by `job`, but it fails. Run it **line by line**, read each error, and fix it.

```
BankWages %>%  
  group_by(Job) %>%  
  summarize(mean_ed = mean(eductaion))
```

Hints:

1. R is case-sensitive — is `Job` really the column name? (`glimpse()` it.)
2. Read the message carefully: does `eductaion` exist?
3. Fix one problem at a time and re-run — don't guess at all of them at once.

05

Introduction to R Markdown

- 01 Recap & Agenda
- 02 Code Hygiene
- 03 Using the Right Function for the Job
- 04 Debugging
- 05 Introduction to R Markdown**
- 06 Recap

What Is R Markdown?

An **R Markdown** file (`.Rmd`) is a plain-text document that weaves together three things in one place:

- **narrative text**,
- **R code**, and
- the **results** that code produces — tables, numbers, and plots.

You *knit* the `.Rmd` into a finished document — **PDF**, **Word**, or **HTML** — with the code re-run every time.

Why Bother?

Because the results are generated **from the code**, they can never drift out of sync with it.

- No pasting a number into your write-up and forgetting to update it when the data changes.
- No loading last week's figure by accident.
- Re-knit, and the whole document reflects the current data.

This is **reproducibility** — and it's exactly the *final draft* to your script's *scrap paper*. (These very slides were built from an R Markdown–style document.)

R Markdown in the Workflow

The pattern we've been building toward:

1. **Explore and draft** your analysis in an R **script** — the messy, iterative part.
2. Once the code works, move it into **R code chunks** in an [.Rmd](#).
3. **Knit** to produce a clean report your reader — or your mentor — can actually read.

Code first, document second. It keeps knitting painless (more on that shortly).

Creating an `.Rmd`

In RStudio:

1. **File** → **New File** → **R Markdown...**, and choose **PDF** as the output.
2. Enter a title (e.g. *ECON 345 Week 4*) and your name, then click **OK**.
3. Save it with a clear name, e.g. `week_04.Rmd`.
4. Press the **Knit** button to build the document.

RStudio opens a small example file — a good template to edit rather than start from a blank page.

The Three Parts of an `.Rmd`

Knitting stitches three kinds of content together:

1. **YAML header** — the block fenced by `---` at the very top. Sets the title, author, date, and output format.
2. **Markdown text** — prose with light formatting: `# Heading`, `_italics_`, `**bold**`, lists.
3. **R code chunks** — fenced blocks of R, opened with ````{r}` and closed with `````, that `knitr` runs and renders.

Code Chunks

Anything you'd put in an R script goes inside a **code chunk**:

```
```${r}  
BankWages %>% tabyl(job)
```
```

- Chunks open with ````${r}` and close with three backticks.
- Insert one fast with **Ctrl+Alt+I** (Cmd+Option+I on a Mac).
- Just like a script, **chunk order matters** — the document runs top to bottom.

Comment vs. Section

One thing that trips people up: `#` means different things in the two parts of an `.Rmd`.

- **Inside a code chunk**, `#` is an R **comment** — same as in a script.
- **In the markdown text**, `#` is a **section heading**, not a comment.

To comment out *markdown* text, wrap it: `<!-- like this -->`.

Chunk Options

You control what a chunk *does* and *shows* with **options** set inside the `{r ...}` header:

| Option | Effect |
|---------------------------------------|--|
| <code>echo = FALSE</code> | run the code, but hide it (show only output) |
| <code>eval = FALSE</code> | show the code, but don't run it |
| <code>include = FALSE</code> | run it, but hide code <i>and</i> output |
| <code>message, warning = FALSE</code> | suppress messages / warnings |
| <code>fig.align, out.width</code> | position and size a plot |

Set them per chunk, or set defaults for the whole document once at the top with `knitr::opts_chunk$set(...)`.

Tables That Belong in a Report

Console output is fine while you work, but a report wants a clean table. `knitr::kable()` takes a data object — like a `summarize()` result — and formats it.

First, the summary itself:

```
ed_by_job <- BankWages %>%  
  group_by(job) %>%  
  summarize(mean_ed = mean(education),  
            median_ed = median(education),  
            n = n())  
ed_by_job
```

```
# A tibble: 3 × 4  
  job      mean_ed median_ed    n  
<ord>    <dbl>    <dbl> <int>  
1 custodial  10.2        12     27  
2 admin     12.9        12    363  
3 manage    17.2        17     84
```

From Data Frame to Table with `kable()`

Pipe that same object into `kable()` — rounding, a caption, and readable column names turn it into report-ready output when you knit:

```
ed_by_job %>%  
  kable(digits = 2,  
        caption = "Years of education by job family",  
        col.names = c("Job", "Mean", "Median", "N"))
```

`kable()` is the simplest option; packages like **kableExtra**, **vtable**, and **stargazer** add styling and ready-made summary tables on top of it.

When Knitting Fails

Because an `.Rmd` is code *and* text, it won't knit if the code doesn't run. Three habits keep that from ruining your day:

1. **Develop your code in a script first**, then move working code into chunks.
2. Set `error = TRUE` to knit *through* an error while you debug.
3. **Restart your R session** and knit clean when things get weird.

Develop in a Script First

The single most reliable way to avoid knitting problems: get the code working in a plain R **script**, then transfer it into chunks.

- Scrap paper (script) → final draft (`.Rmd`).
- Moving already-working code into chunks means the only new problems are formatting ones, not logic ones.

error = TRUE and a Clean Session

By default a chunk error **stops** the knit. Setting `error = TRUE` tells knitr to print the error and keep going — useful for seeing *all* the problems at once:

```
# At the top of the document: allow every chunk to knit through errors
knitr::opts_chunk$set(echo = TRUE, error = TRUE)
```

And when results look stale or impossible, **restart R** (Session → Restart R) and knit from a clean slate — an `.Rmd` should stand on its own, with no leftovers from your console.

R Markdown and Your Project

R Markdown is the reporting layer for the work you're already doing. Your project updates and write-ups live here:

- Draft and test the analysis in a **script**.
- Assemble the narrative, code, and results in an **.Rmd**.
- **Knit** to the format you need — Word for project updates, PDF for a paper.

Because the tables and figures are generated from the code, revising your analysis and refreshing the document is a single **re-knit** — no re-pasting, no stale numbers.

This week: create an **.Rmd**, knit it, and use `kable()` to present a summary table from your project data.

06

Recap

- 01 Recap & Agenda
- 02 Code Hygiene
- 03 Using the Right Function for the Job
- 04 Debugging
- 05 Introduction to R Markdown
- 06 Recap**

Recap

Today was about writing code a reader can follow:

- **Code hygiene** — organize a script top to bottom, comment the *why*, name for clarity, and test on a copy before overwriting
- **The right function for the job** — `case_when()` replaces the endless sea of nested `if_else()` for multi-way conditions
- **Debugging** — read the message, run a pipe line by line, and mind the difference between a warning and an error
- **R Markdown** — knit text, code, and results into one reproducible document, and format tables with `kable()`

Draft in the script, present in the `.Rmd`.

Links & Further Reading

- [R for Data Science, Ch. 5: Data Transformation — conditional logic](#)
- [dplyr::case_when\(\) documentation](#)
- [R Markdown Cheat Sheet](#)
- [The tidyverse style guide \(naming, spacing, structure\)](#)

Coming up: data visualization with `ggplot2` — turning these summaries into charts.